

EFFICIENZA ALGORITMI

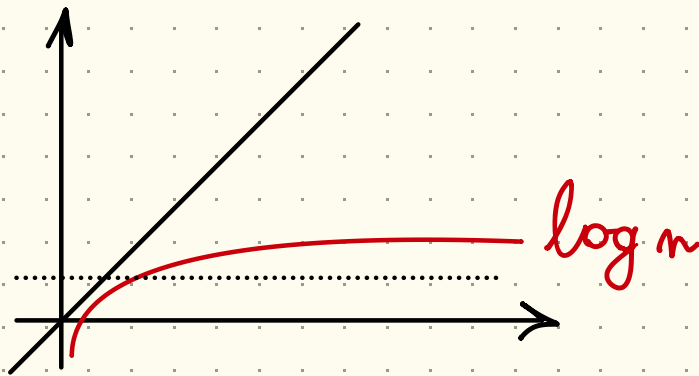
• BINARY SEARCH

• PASSAGGI

ELEMENTI	PASSAGGI	SCANSIONE
16	18	32
100	26	200
1000	38	2000
10'000	54	20000
40'000	62	80000

$\overset{\text{TETHA}}{\downarrow}$
 $H(\log n)$
 $H(n)$

• COMPLESSITA'



• NUMERI CASUALI IN C++

INIZIALIZZAZIONE: `SRAND(TIME(NULL))`

SONO NUMERI PSEUDOCASUALI

PER GENERARE: `RAND()`

SE VOGLIO GENERARE UN n COMPRESO
IN UN RANGE

es) GENERARE n CASUALE TRA 0-48

$$\Downarrow \\ \text{RAND()} \% 49$$

es) GENERARE n CASUALE TRA 19-48

$$\Downarrow \\ (\text{RAND()} \% 30) + 19$$

REGOLA GENERALE

$$\text{INT NCASUALE} = \text{INIZIO} + \text{RAND()} \% (\text{FINE} - \text{INIZIO} + 1)$$

• QUICK SORT

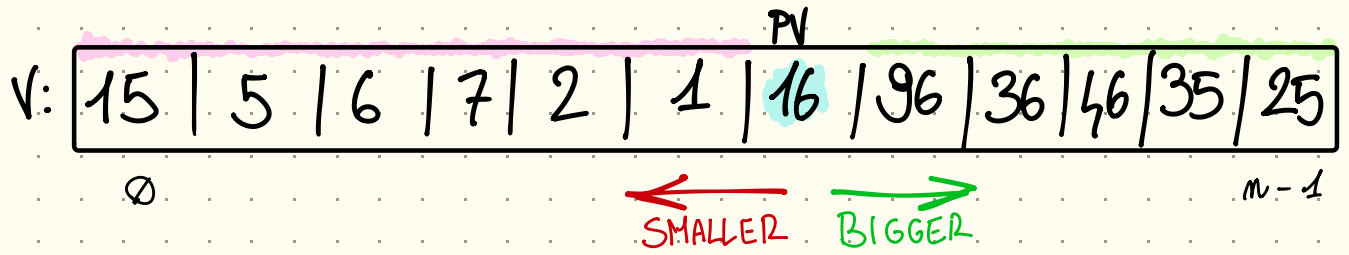
V:

25	35	15	5	46	36	16	96	6	7	2	1
----	----	----	---	----	----	----	----	---	---	---	---

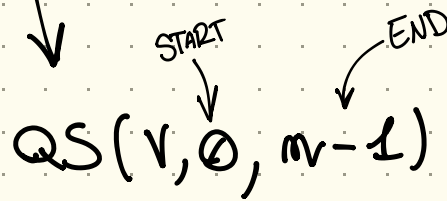
0 $n-1$

1. SCELTA CASUALE DI UN NUMERO
2. CON LA FUNZIONE PARTIZIONA, TUTTI I NUMERI ALLA SUA SINISTRA SARANNO PIÙ PICCOLI, ALLA SUA DESTRA PIÙ GRANDI
NON RIORDINA!! PARTIZIONA!!

b) DOPO LA CHIAMATA A PARTIZIONA



3. CHIAMO QS SULLE DUE SEZIONI, ESCLUSO PV



• COMPLESSITA'

NEL CASO PEGGIORE (SCEGLIE SEMPRE PRIMO O ULTIMO ELEMENTO) HA COMPLESSITA' QUADRATICA (COME IL SELECTION SORT)

⇓

n^2

NEL CASO MIGLIORE (SCEGLIE LA MEDIANA) HA COMPLESSITA' MIGLIORE DEL MERGE SORT

⇓

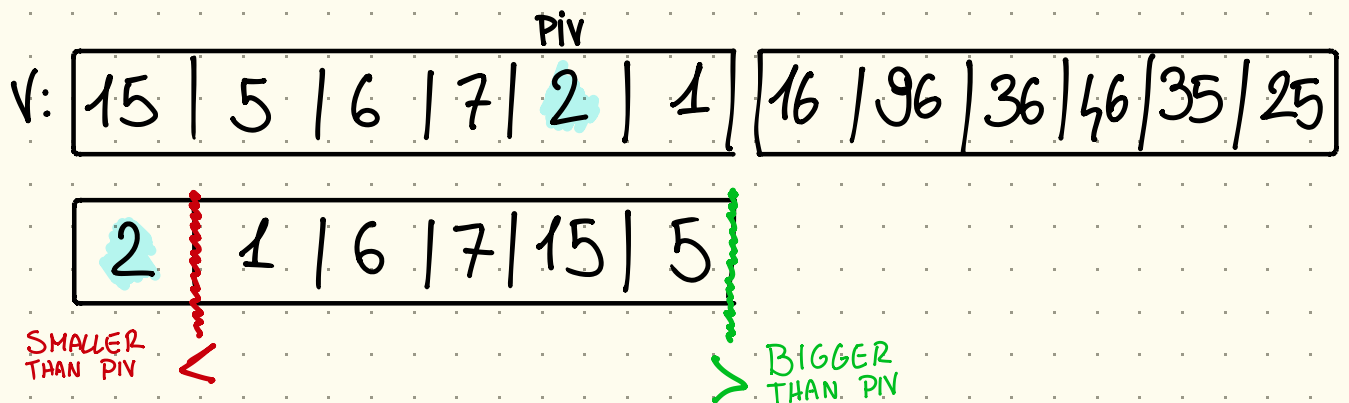
$n \log n$

PER MIGLIORARE LA COMPLESSITA', EVITANDO CHE SCELGA UN NUMERO TROPPO GROSSO O TROPPO PICCOLO



SI ESTRAGGONO CASUALMENTE 3 NUMERI E SI SCEGLIE IL MEDIANO TRA I 3

4. SCEGLIE DI NUOVO IL PV NELLE DUE SEQUENZE



5. CREA DEI "MURI" DI PIU' PICCOLI E PIU' GRANDI (USANDO 2 INDICI, i E j)